DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Design and Evaluation of Multi-Agentic AI Systems for Contract Lifecycle Management SaaS Platforms

Nguyen, Duc Trung, B.Sc.

DEPARTMENT OF INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Master's Thesis in Informatics

Design and Evaluation of Multi-Agentic AI Systems for Contract Lifecycle Management SaaS Platforms

Entwurf und Evaluation multiagentenbasierter KI-Systeme für Contract-Lifecycle-Management-SaaS-Plattformen

Author: Nguyen, Duc Trung, B.Sc.
Supervisor: Prof. Dr. Ingo Weber
Advisor: Samed Bayer, M.Sc.
Submission Date: 04.05.2026

I confirm that this master's thesis in informatics is my own work and I have documented all sources and material used.

Munich, 04.05.2026 Nguyen, Duc Trung, B.Sc.

Acknowledgments

Abstract

Contract Lifecycle Management platforms increasingly rely on large language models to automate clause extraction from commercial contracts, and both research and vendor communities have advocated multi-agent architectures as a superior alternative to single-prompt deployments. The empirical case for this advocacy is, however, thin: existing legal multi-agent systems each evaluate a single architecture on a single benchmark, no published work has compared multi-agent patterns to single-agent baselines across multiple frontier-model families, and recent cross-domain work suggests that multi-agent gains diminish as base-model capability rises.

This thesis presents the first controlled cross-architecture cross-model comparison of multi-agent designs for clause extraction on the CUAD benchmark. Five configurations spanning zero-shot, chain-of-thought, pipeline decomposition, LLM-routed specialists, and a consolidated single-agent ablation are evaluated across eleven frontier models from three providers, yielding 163,644 paired predictions analysed with paired statistical tests at the natural class distribution.

No multi-agent configuration consistently outperforms the strongest single-agent baseline. Pooled accuracy across the eleven models falls within a one-point band, and architectural ranking varies with model scale rather than model family: larger models perform best under simpler baselines, while lighter models gain from structured decomposition. Two mechanisms explain the null result. The pipeline architecture exhibits a *classification-gate rejection cascade* — its upstream classifier silently overrejects on some models — and the LLM router in the specialist architecture is empirically redundant: it converges to a deterministic category lookup and produces lower-quality span outputs than the consolidated ablation. Multi-agent configurations carry a 24%–70% cost premium without aggregate accuracy benefit.

These results delineate when multi-agent designs help, are inert, or actively hurt in enterprise CLM, and motivate a practitioner-oriented analysis of the gap between benchmark performance and production deployment.

Contents

Acknowledgments **iv**

Abstract **v**

1 Introduction **1**
1.1 Motivation . . . 1
1.2 Research Gap . . . 1
1.3 Research Questions . . . 2
1.4 Hypotheses . . . 3
1.5 Contributions . . . 3
1.6 Thesis Outline . . . 4

2 Background and Related Work **5**
2.1 Background . . . 5
2.1.1 Contract Lifecycle Management . . . 5
2.1.2 Large Language Models . . . 5
2.1.3 Agentic AI and Multi-Agent Systems . . . 6
2.1.4 The CUAD Benchmark . . . 6
2.2 Contract Analysis Methods . . . 7
2.3 Multi-Agent Legal AI . . . 7
2.4 Conditions for Multi-Agent Effectiveness . . . 8
2.5 Evaluation Methodology . . . 9
2.6 Research Gap . . . 10

3 Methodology **11**
3.1 Research Design . . . 11
3.2 Dataset and Sampling . . . 11
3.3 Metrics . . . 12
3.4 Hypotheses . . . 13
3.5 Statistical Methodology . . . 13
3.6 Model Selection . . . 14
3.7 Threats to Validity . . . 14

4 System Design and Implementation **16**
4.1 Architectural Patterns Evaluated 16
4.2 Baseline Architectures 16
4.3 Multi-Agent Architectures 17
4.4 Implementation Stack 18
4.5 Traceability and Auditability 20
4.6 Experimental Infrastructure 21

5 Results **23**
5.1 Run Coverage and Sample Accounting 23
5.2 H1 — Multi-Agent vs. Single-Agent Baselines 24
5.3 H2 — Rare-Category Gap 25
5.4 H3 — Architecture vs. Prompts (M1 vs. M6) 26
5.5 Additional Findings 27
5.5.1 The Classification-Gate Rejection Cascade 27
5.5.2 Cost-Efficiency Analysis 28

6 Discussion **31**
6.1 When Multi-Agent Helps: Evidence for Pipeline Decomposition 31
6.2 When Multi-Agent Hurts: The LLM-Router Problem 31
6.3 Architecture vs Prompts: The H3 Verdict 31
6.4 Practitioner Perspective: Gaps Between Benchmarks and Real-World CLM 31
6.5 Connecting to Prior Work 31
6.6 Limitations 31

7 Conclusion and Outlook **32**
7.1 Summary of Contributions 32
7.2 Answers to Research Questions 32
7.3 Outlook and Future Work 32

List of Figures **33**

List of Tables **34**

Bibliography **36**

1 Introduction

1.1 Motivation

Contract Lifecycle Management (CLM) is the workflow through which an organisation authors, negotiates, executes, monitors, and retires its contractual instruments. The economic surface is large: organisations forfeit value equivalent to roughly 9% of annual revenue through poor contracting practices, with contract data dispersed across an average of 24 systems and 83% of executives reporting their contracts too rigid to accommodate change [Wor25]. Goldman Sachs estimate 44% of U.S. legal-task time is technically automatable by generative AI — second-highest exposure across all professions [Hat+23].

Frontier large language models (LLMs) match junior lawyers on issue-identification F-scores (0.87 vs. 0.88) while reducing review time by $\approx 100\times$ [Mar+24], and independent benchmarks confirm competitiveness on standardised clause-extraction tasks [Hen+21; Liu+25]. Single-prompt deployments nonetheless dominate commercial CLM, and three production failures persist: hallucination, prompt sensitivity [Cha+24], and laziness — the silent default to "no related clause" even when the clause is present in the contract [Liu+25].

Multi-agent architectures are the proposed remedy. Decomposing clause analysis into specialised agents promises improved recall through specialisation, improved explainability through inspectable inter-agent traces, and addresses transparency and traceability requirements under Article 13 of the EU AI Act [EC24] and Article 22 of the GDPR [EC16]. Frameworks including AutoGen [Wu+23], MetaGPT [Hon+24], and Mixture-of-Agents [Wan+24] have made such architectures cheap to assemble. Whether they actually improve contract analysis is, however, an open empirical question.

1.2 Research Gap

Three observations motivate the gap this thesis addresses.

First, existing legal multi-agent systems — ChatLaw [Cui+24], PAKTON [Rap+25], L-MARS [WY25] — each evaluate a single architecture on a single benchmark without ablating the architectural choice itself. The empirical question of which multi-agent pattern is preferable for clause-level contract analysis under a fixed model and task remains empirically unresolved.

Second, recent cross-domain work questions whether multi-agent designs help at all once base-model capability is high. Gao et al. show diminishing returns for frontier models whose long-context reasoning absorbs the workload that previously required decomposition [Gao+25]. Cemri et al. construct a 14-mode failure taxonomy across seven multi-agent frameworks and report only modest 14–15% recovery from simple interventions, suggesting structural rather than prompt-level causes [Cem+25]. Bogavelli et al. find no universally optimal architecture across frontier models on enterprise tasks, with model–architecture interaction dominating absolute architectural ranking [Bog+25].

Third, prior CUAD evaluations have used inconsistent sampling designs. Evaluations at inflated positive ratios produce optimistic precision estimates because few absent samples are tested, masking the dual failure mode of over-extraction.

No published study simultaneously (i) cross-varies multi-agent architecture and frontier-model family on CUAD, (ii) samples at the natural class distribution, (iii) applies paired statistical tests with multiple-comparison correction, and (iv) ablates the multi-agent design against a consolidated single-agent prompt. This thesis addresses all four requirements.

1.3 Research Questions

Main RQ. Under what conditions do multi-agent architectures improve contract analysis accuracy, efficiency, and explainability in enterprise CLM systems, and which architectural patterns are most effective?

RQ1. Under what model-family and prevalence conditions does each architectural pattern — pipeline decomposition, specialist routing, or consolidated prompting — achieve the highest F_2?

RQ2. How do multi-agent configurations compare to single-agent baselines in F_2 and provider cost across clause categories and contract lengths?

RQ3. Which architectural decisions produce auditable reasoning traces, and what tradeoffs do they introduce?

1.4 Hypotheses

H1. At least one multi-agent configuration outperforms the strongest single-agent baseline on F_2 (McNemar exact, α=0.05, BH-corrected).

H2. Multi-agent configurations reduce the rare–common F_2 gap relative to single-agent baselines (paired bootstrap on gap difference, 95% CI).

H3. The LLM-routed specialist system M1 is statistically distinguishable from the consolidated specialist ablation M6 on F_2 (two-sided McNemar exact).

RQ3 is addressed by system design rather than statistical hypothesis test; the auditability properties of the implementation are documented in §4.5.

1.5 Contributions

C1 **Cross-architecture benchmark.** The first cross-family cross-architecture comparison on CUAD: 5 architectural configurations $\times$ 11 frontier models, yielding 163,644 paired sample-level predictions analysed with McNemar exact tests, paired bootstrap, and Benjamini–Hochberg correction.

C2 **Model-scale gradient.** Architectural ranking is not constant across the model panel: larger models achieve their best F_2 under simpler single-agent baselines, while lighter or weaker models gain from structured decomposition. No architecture is universally optimal — pooled F_2 across all five configurations spans a 0.011-point band.

C3 **Classification-gate cascade.** A failure mode of pipeline-decomposition multi-agent architectures, named and quantified: the upstream classifier accounts for the majority of false negatives, propagating silent rejections downstream. The cascade is severe — Gemini 2.5 Flash's wrong-rejection rate jumps from 2.7% under a single-agent baseline to 40.9% in the pipeline — and its direction depends on the base model's laziness profile, harming low-laziness models and helping high-laziness ones, making the effect predictable from a single-agent baseline run.

C4 **Routing redundancy with span-quality degradation.** The LLM router in the specialist architecture is empirically redundant: it agrees with a deterministic category-to-specialist mapping on every model in the panel — 100% routing accuracy on 10 of 11 models — and the consolidated single-agent ablation matches it on F_2 while producing cleaner spans. The routing topology adds cost without architectural benefit.

A methodological observation underpins all four contributions. The thesis samples at CUAD's natural 30%/70% positive/negative class ratio rather than the inflated positive ratios used in some prior work, exposing over-extraction as the dominant zero-shot failure mode that positive-heavy sampling masks (§3.2).

1.6 Thesis Outline

Chapter 2 surveys background and related work. Chapter 3 specifies methodology. Chapter 4 documents the implementation. Chapter 5 reports empirical results. Chapters 6 and 7 discuss findings and conclude.

2 Background and Related Work

2.1 Background

2.1.1 Contract Lifecycle Management

CLM is the end-to-end workflow through which an organisation authors, negotiates, executes, monitors, and retires its contractual instruments. Figure 2.1 shows the canonical six-stage model. The economic surface is large: $\approx$9% annual revenue loss from poor contracting [Wor25], with 44% of U.S. legal-task time exposed to automation [Hat+23]. Three NLP sub-tasks dominate the Execute and Monitor stages: *clause extraction* (locate and bound the span instantiating a clause category), *risk identification* (classify extracted clauses against playbook standards), and *obligation tracking* (convert executed clauses into a structured event calendar).

2.1.2 Large Language Models

An LLM is a neural language model parameterised by θ that factorises a token sequence $\mathbf{x} = (x_1, \ldots, x_n)$ autoregressively:

$$P(\mathbf{x}; \theta) = \prod_{t=1}^{n} P(x_t \mid x_1, \ldots, x_{t-1}; \theta). \tag{2.1}$$

Training minimises the negative log-likelihood $\mathcal{L}(\theta) = -\sum_{t=1}^{n} \log P(x_t \mid x_{<t}; \theta)$ over large text corpora. At inference the model generates x_t by sampling from the distribution over the vocabulary conditioned on the preceding context. Instruction tuning and reinforcement learning from human feedback (RLHF) align the base model toward task-following behaviour without changing the fundamental factorisation [Dev+19].

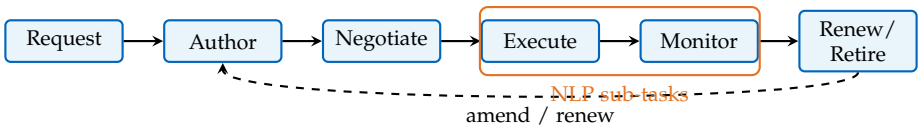


Figure 2.1: CLM lifecycle. Clause extraction, risk identification, and obligation tracking (highlighted) occur primarily in the Execute and Monitor stages.

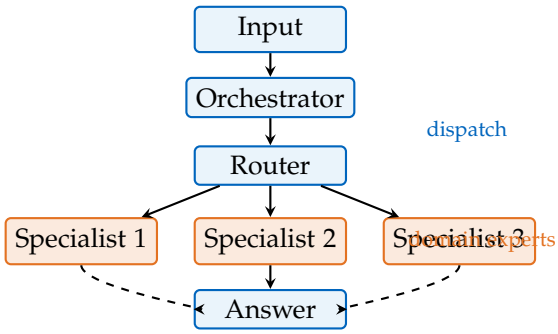


Figure 2.2: Canonical hierarchical multi-agent architecture. The orchestrator coordinates; the router dispatches to one specialist per request; specialists return typed answers. Dashed arrows indicate conditional paths.

2.1.3 Agentic AI and Multi-Agent Systems

An *agent* is an LLM invocation that (i) receives a structured role specification and context, (ii) may call tools or sub-agents, and (iii) emits a typed output. A *multi-agent system* (MAS) is a directed graph of agents with typed message-passing edges; see Figure 2.2. Key structural dimensions are topology (hierarchical vs. peer), routing (static pipeline vs. LLM-mediated dispatch), specialisation granularity (per-task vs. per-domain vs. per-step), and verification structure (none, reflective, debate, or judge-agent) [Wu+23; Hon+24; Li+23].

2.1.4 The CUAD Benchmark

CUAD [Hen+21] is the standard public benchmark for commercial-contract clause extraction (Table 2.1). Each (contract, category) pair is labelled with zero or more verbatim attorney-annotated spans; absence of a span is a true negative. Category prevalence is heavily skewed toward absence ($\approx$70% negative at the instance level), which is why sampling design is a methodological lever (Section 3.2).

Property	Value
Contracts	510
Clause categories	41
Expert annotations	>13,000 spans
Source	EDGAR commercial filings
Train / test split	399 / 102
Task formulation	Span extraction + binary presence
License	CC BY 4.0
Long contracts (>100k chars)	81 (used in this thesis)

Table 2.1: CUAD dataset statistics [Hen+21].

2.2 Contract Analysis Methods

Rule-based and statistical methods. Pre-LLM extraction used regular expressions, gazetteers, and CRFs [LMP01]. High precision on narrow clause types degraded on out-of-template contracts. BiLSTM-CRF [Lam+16] improved cross-document robustness; BERT-class encoders [Dev+19] then replaced recurrent encoders with self-attention, enabling fine-tuning on small legal corpora. LEGAL-BERT [Cha+20] showed domain pre-training improves downstream legal NLP performance.

LLM-based methods. BERT-class models are limited to 512 tokens, require labelled data per clause category, and do not generalise zero-shot. LLMs address all three limitations. Martin et al. [Mar+24] report GPT-4-1106 at $F = 0.87$ on issue determination, matching legal-process outsourcers at $\approx 1/100$ the cost (vendor-funded; directional finding replicated independently [Liu+25]). ContractEval [Liu+25] benchmarks 19 LLMs on CUAD; proprietary models outperform open-source, scaling yields diminishing returns for open-source models, and reasoning-mode prompting improves span quality but degrades binary correctness. Open-source models exhibit a *laziness* failure: defaulting to "no related clause" even when the clause is present. Table 2.2 compares the main contract-analysis benchmarks.

2.3 Multi-Agent Legal AI

Three multi-agent legal AI systems are directly relevant. Table 2.3 codes them and the present configurations on five dimensions.

Dataset	Task	Size	Categories	Used here
CUAD [Hen+21]	Span extraction	510 contracts	41	✓
ContractNLI [KM21]	Doc-level NLI	607 NDAs	17	—
MAUD [Wan+23]	Reading comp.	152 M&A agmt.	152	—
LEDGAR [Tug+20]	Multi-label cls.	60k contracts	100+	—

Table 2.2: Main contract-analysis benchmarks.

System	Domain	Topology	Routing	Specialisation	Verification	Benchmark	Ablated?
ChatLaw v2 [Cui+24]	Legal QA (ZH)	Hierarchical	SOP	Per-task	MoE consensus	LawBench	No
PAKTON [Rap+25]	Contract QA	Pipeline	Static	Per-step (3)	ReAct iter.	ContractNLI	No
L-MARS [WY25]	Legal search	Pipeline	Static	Per-step (4)	Judge agent	LegalSearchQA	No
AutoGen [Wu+23]	General	Peer	Conversational	Per-task	None	HumanEval	N/A
MetaGPT [Hon+24]	Software dev.	Hierarchical	SOP	Per-role	Review agent	SWE-Bench	No
MoA [Wan+24]	General	Layered	Broadcast	None	Aggregator	MMLU	No
B1 / B4 (ours)	CLM / CUAD	Single-agent	None	None / CoT	None	CUAD	✓
M0 (ours)	CLM / CUAD	Pipeline	Static gate	None	None	CUAD	✓
M1 (ours)	CLM / CUAD	Hierarchical	LLM	Per-domain (3)	None	CUAD	✓
M6 (ours)	CLM / CUAD	Single-agent	None	Union prompt	None	CUAD	✓

Table 2.3: Multi-agent systems for legal AI coded on five architectural dimensions. All prior systems evaluate a single architecture without an ablation.

Reported gains must be interpreted carefully: ChatLaw v2's +7.73-point improvement is relative to GPT-4 without a multi-agent ablation; L-MARS's +38 points is against zero-shot on a self-constructed benchmark. Neither isolates the multi-agent design from retrieval or prompting choices made in the same system.

2.4 Conditions for Multi-Agent Effectiveness

Three recent studies directly bound the scope of multi-agent benefits. Gao et al. [Gao+25] show the advantage diminishes for frontier models (o3, Gemini 2.5 Pro) whose long-context reasoning absorbs workloads that previously required decomposition. Cemri et al. [Cem+25] identify 14 MAST failure modes in >1,600 traces across 7 frameworks; simple interventions yield only 14–15% recovery, suggesting structural rather than prompt-level causes. Bogavelli et al. [Bog+25] (AgentArch: 18 configurations, 6 frontier models) find no universally optimal architecture — best-of-class reaches only 35.3% on harder enterprise tasks — and ReAct-style prompting produces hallucinated tool calls in multi-agent settings. Sanwal [San25] proposes external verification at each

reasoning layer to address unverified intermediate steps in chain-of-thought pipelines. Taken together: multi-agent designs help when tasks are decomposable, specialists meaningfully outperform a generalist, and the underlying model is weak enough that decomposition provides a lever. This thesis empirically tests whether these conditions hold for clause-level CUAD extraction.

2.5 Evaluation Methodology

Metrics. Let TP, FP, FN, TN be per-sample counts, S the predicted span, T the reference span. Formal definitions:

$$F_\beta = \frac{(1+\beta^2)\,\mathrm{TP}}{(1+\beta^2)\,\mathrm{TP} + \beta^2\,\mathrm{FN} + \mathrm{FP}}, \quad \beta \in \{1, 2\} \tag{2.2}$$

$$J(S,T) = \frac{|S \cap T|}{|S \cup T|} \tag{2.3}$$

$$\lambda = \frac{|\{i : y_i{=}1,\ \hat{y}_i{=}0\}|}{|\{i : y_i{=}1\}|} \qquad \text{(laziness rate)} \tag{2.4}$$

$$\omega = \frac{|\{i : y_i{=}0,\ \hat{y}_i{=}1\}|}{|\{i : y_i{=}0\}|} \qquad \text{(over-extraction rate)} \tag{2.5}$$

$\beta{=}2$ weights recall four times as heavily as precision, reflecting the legal-risk asymmetry where missed clauses incur downstream financial exposure [Hen+21; Liu+25]. Jaccard is reported at span level conditional on extraction.

Statistical testing. Dror et al. [Dro+18] is the canonical reference. For paired binary outcomes McNemar's exact test [McN47] is used; for continuous metrics the paired non-parametric bootstrap [Efr79; RM05] with 95% percentile CI. False discovery rate is controlled via Benjamini–Hochberg [BH95] across the 10 pairwise configuration comparisons ($\alpha{=}0.05$). A single sampling design and prompt template is held fixed across all models; prompt sensitivity is treated as a validity threat (Section 3.7).

Sampling. CUAD's natural clause-presence rate is $\approx 30\%$ positive at the instance level. Evaluations that inflate this ratio produce optimistic precision estimates [Liu+25]. This thesis samples at the natural ratio, exposing over-extraction as a primary failure mode.

2.6 Research Gap

No published study simultaneously (i) cross-varies multi-agent architecture and frontier-model family on CUAD, (ii) samples at the natural class distribution, (iii) applies paired statistical tests with multiple-comparison correction, and (iv) ablates the multi-agent design against a consolidated single-agent prompt. This thesis addresses all four requirements.

3 Methodology

3.1 Research Design

Five configurations are evaluated against 11 frontier LLMs on 81 long CUAD contracts (>100,000 chars), yielding $5 \times 11 = 55$ official runs and 163,644 analysed predictions. Differences between configurations are minimised to a single architectural axis per comparison (Table 4.1), following the controlled comparative protocol of [Dro+18; Car+20]. Every statistical test is paired on the intersection of sample IDs so performance differences are attributable to architecture or model, not dataset drift.

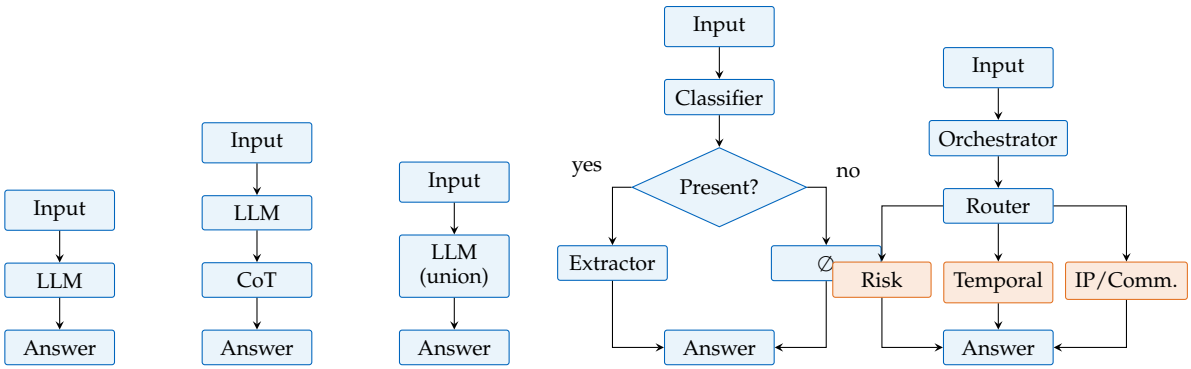
ID	Name	Stages	Routing	Specialisation
B1	Zero-shot single agent	1	None	Generalist
B4	Chain-of-thought single agent	1	None	Generalist + CoT
M0	Classify-then-extract pipeline	2	Static gate	Generalist
M1	Orchestrator + 3 specialists	2	LLM router	Three domain experts
M6	Consolidated specialist (ablation)	1	None	Union of M1 prompts

Table 3.1: Five evaluated configurations. M0 vs. B1 isolates the classification gate; M1 vs. M6 isolates routing and decomposition; B4 vs. B1 isolates chain-of-thought.

Figure 3.1 shows the message-flow diagram for each configuration.

3.2 Dataset and Sampling

CUAD [Hen+21] is the evaluation corpus (see Table 2.1 for statistics). We restrict to 81 contracts >100,000 characters because long-context performance is the regime where decomposition is hypothesised to add value, and long contracts approximate the operational distribution of CLM platforms. The 81-contract subset yields $81 \times 41 = 3{,}321$ raw (contract, category) pairs per run.



(a) B1: Zero-shot (b) B4: Chain-of-thought (c) M6: Ablation (d) M0: Classify-then-extract (e) M1: Router + specialists

Figure 3.1: Message-flow diagrams for the five configurations. Input = contract text + clause-category indicator. Blue nodes: generalist LLM invocations. Orange nodes: domain-specialist prompts. ∅ = silent no-clause return.

Sampling strategy. A `neg_ratio=0.7` sampler produces a natural 30%/70% positive/negative split per run, mirroring CUAD's actual class prior. Prior work with inflated positive ratios (≈67%) masks over-extraction because few absent instances are evaluated; the natural prior exposes it directly. Sampling is seeded (`seed=42`) so the same (contract, category) pairs are evaluated across all configurations, making every statistical test paired at the instance level. Each run yields ≈2,999 predictions; 1,297 of 164,941 total (0.79%) were excluded for malformed classification tokens.

Prevalence tiers. The 41 CUAD categories are partitioned into three tiers — *common*, *moderate*, *rare* — by natural positive rate in the 81-contract subset. Tier cutpoints are fixed before any model is run.

3.3 Metrics

All metrics are defined in Section 2.5 (Equations 2.2–2.5). F_2 is the primary hypothesis-testing metric (β=2, recall weighted 4× over precision). F_1 is reported as a secondary comparison-friendly metric. Jaccard similarity (Eq. 2.3) measures span quality conditional on extraction. Laziness (λ, Eq. 2.4) and over-extraction (ω, Eq. 2.5) are diagnostic rates. All metrics are reported pooled and per prevalence tier.

3.4 Hypotheses

H1. $\exists C \in \{\text{M0}, \text{M1}, \text{M6}\}$ such that C outperforms $B^\star \in \{\text{B1}, \text{B4}\}$ on F_2. Null: no such C exists.

H2. $\Delta\text{Gap}(C_{\text{multi}}, B^\star) < 0$ where $\text{Gap}(C) = F_2^{\text{common}}(C) - F_2^{\text{rare}}(C)$. Null: $\Delta\text{Gap} \geq 0$.

H3. M1 and M6 are statistically distinguishable on F_2. Null: $\Delta F_2(\text{M1}, \text{M6}) = 0$.

3.5 Statistical Methodology

McNemar's exact test. For configurations A and B evaluated on the same instances, let $b = |\{i : A_i \text{ correct}, B_i \text{ wrong}\}|$ and $c = |\{i : A_i \text{ wrong}, B_i \text{ correct}\}|$. The two-sided exact p-value is:

$$p = 2\,\mathbb{P}\big[\text{Bin}\big(b{+}c, \tfrac{1}{2}\big) \leq \min(b, c)\big] \tag{3.1}$$

implemented via `scipy.stats.binomtest` [Vir+20]. The exact form is preferred over the χ^2 approximation because discordant counts are small on rare-tier subsets [McN47].

Paired bootstrap. For continuous metrics (F_2, F_1, Jaccard, gap differences), Algorithm 1 gives the paired bootstrap CI [Efr79; RM05].

Algorithm 1 Paired bootstrap confidence interval for ΔF_2

Require: Predictions $\hat{y}^A$, $\hat{y}^B$, labels y, B=300, α

```
for b = 1 to B do
    I ← sample n indices with replacement from {1, ..., n}
    δ_b ← F_2(ŷ^A_I, y_I) − F_2(ŷ^B_I, y_I)
end for
return [percentile(δ, α/2), percentile(δ, 1−α/2)]
```

Multiple-comparison correction. The H1 family contains $\binom{5}{2} = 10$ pairwise comparisons per model. Benjamini–Hochberg [BH95] controls FDR at α=0.05: reject $H_0^{(i)}$ if

$$p_{(i)} \leq \frac{i}{m}\,\alpha, \quad p_{(1)} \leq \cdots \leq p_{(m)},\ m = 10. \tag{3.2}$$

Raw and BH-corrected p-values are both reported. Intersection of sample IDs is taken symmetrically when partial runs differ in coverage; intersection size is reported with each test.

3.6 Model Selection

Table 3.2 lists the 11 frontier proprietary models. Open-weights models are excluded from the main analysis because commercial CLM platforms deploy proprietary APIs; an open-weights extension is planned (Chapter 7).

Family	Model	Tier
Anthropic	Claude Haiku 4.5	Light
	Claude Sonnet 4	Mid
	Claude Sonnet 4.6	Mid
	Claude Opus 4.6	Heavy
Google	Gemini 2.5 Flash	Light
	Gemini 2.5 Pro	Heavy
	Gemini 3 Flash	Light
	Gemini 3.1 Pro	Heavy
OpenAI	GPT-4.1	Mid
	GPT-4.1 Mini	Light
	GPT-4.1 Nano	Light

Table 3.2: Eleven frontier proprietary models evaluated. Three families; compute-tier range covers light-to-heavy within each family.

3.7 Threats to Validity

Internal. Configuration parity is maintained by holding the CUAD loader, sampling seed, prompt registry, and trace schema constant across configurations. M1 and M6 share the same textual specialist instructions (split vs. concatenated), so M1 vs. M6 isolates architecture, not prompt content.

External. Single language (English), single jurisdiction (U.S. commercial contracts), single corpus (CUAD). Model snapshots are pinned; provider updates can shift behaviour.

Construct. F_2 is a proxy for legal review utility, not a direct measurement. Reviewer cost of inspecting a spurious span is workflow-dependent; this is mitigated by reporting precision, recall, λ, and ω separately.

Statistical. Bootstrap percentile CIs under-cover for small samples and skewed statistics [Efr79]; p-values are treated as decisive only when corroborated by the McNemar exact test. Model identity is a fixed factor; inference does not generalise beyond the 11 models studied.

4 System Design and Implementation

This chapter documents the system that operationalises the configurations of Chapter 3. The objective is reproducibility: the architectures are described at a level of detail sufficient for an independent team to re-implement them, and the supporting infrastructure (prompt registry, model client, observability layer, dashboard, and test suite) is described at the same level. Code references in this chapter point to the thesis artifact released alongside the document.

4.1 Architectural Patterns Evaluated

The five configurations populate three orthogonal axes of the agentic design space: *number of stages* (single vs. multi-stage), *routing* (none, static, or LLM-driven), and *specialisation* (a single generalist agent vs. several domain specialists vs. a consolidated specialist prompt). Table 4.1 summarises the five configurations and their position in this space.

ID	Name	Stages	Routing	Specialisation
B1	Zero-shot single agent	1	none	generalist
B4	Chain-of-thought single agent	1	none	generalist + reasoning
M0	Classify-then-extract pipeline	2	static (gate)	generalist
M1	Orchestrator + 3 specialists	2	LLM router	three domain specialists
M6	Consolidated specialist (ablation)	1	none	one agent, union of specialist prompts

Table 4.1: The five evaluated configurations across the three design axes.

4.2 Baseline Architectures

B1 — ContractEval replication. B1 is a faithful replication of the zero-shot prompt protocol of [Liu+25]. The model receives a single user message containing the contract

text, the target category name, the category indicator (a short attorney-style description of what the category covers, taken from CUAD's annotation guidelines), and an explicit instruction to either return the verbatim span(s) of the clause or the literal string "no related clause". The system message is identical across categories. No examples, no chain-of-thought instructions, and no auxiliary structure are added. B1 establishes the floor for a competent single-agent reading.

B4 — Chain-of-thought single agent. B4 retains B1's task framing but explicitly elicits step-by-step reasoning followed by a structured final answer. The user prompt instructs the model to reason about the category definition, scan the contract for candidate spans, weigh ambiguous matches, and only then emit a final `<answer>...</answer>` block containing either verbatim span(s) or "no related clause".

The B4 prompt went through one substantive revision during pilot work. The original delimiter-based parser extracted any text occurring after a sentinel string, which under certain models produced spurious matches: when the chain-of-thought monologue itself quoted contract passages, the parser harvested them as if they were the final answer, inflating the over-extraction rate. The fix was to enforce a structured-output contract: the parser now requires the final block to be enclosed in an explicit answer tag and rejects the trace otherwise, with a small repair step that re-prompts the model for a structured emission if the tag is missing. Only the post-fix version is in the official runs; the parsing-bug runs are retained for reference but excluded from all reported statistics.

4.3 Multi-Agent Architectures

M0 — Classify-then-extract pipeline. M0 decomposes the per-category task into two LLM calls. The first call is a binary *classifier* agent that receives the contract and the category and returns a single yes/no token indicating whether any span of the category is present. The second call is an *extractor* agent that is invoked only when the classifier returns yes; it receives the same inputs and produces the verbatim span(s). When the classifier returns no, the extractor is bypassed and the system records "no related clause" as the final answer. The two agents share a category indicator but have disjoint prompts: the classifier's prompt explicitly forbids span emission, and the extractor's prompt assumes presence. M0 implements the simplest form of conditional computation in the design space and is the architectural reference for the value of a classification gate.

M1 — Orchestrator with three specialists. M1 consists of an *orchestrator* that receives the contract and the category and routes the request, via an LLM router prompt, to one of three domain specialists. The router prompt enumerates the three specialists, gives a one-line description of each, and asks for a single category-name decision. The three specialists partition the 41 CUAD categories as follows:

- **Risk/Liability specialist** (13 categories): liability caps, indemnification, insurance, warranty, governing law, and similar risk-allocation provisions.
- **Temporal/Renewal specialist** (11 categories): term, renewal, termination, notice periods, post-termination obligations, and other time-bounded provisions.
- **IP/Commercial specialist** (17 categories): IP ownership and licence grants, exclusivity, non-compete, audit rights, and commercial covenants.

Categories were assigned to specialists by the author in consultation with the CUAD category descriptions; the assignment was fixed before any model was run. Each specialist's prompt is a focused reading instruction tuned to its category cluster, including category indicators and asymmetry guidance (e.g. liability caps require recall of monetary thresholds, renewal clauses require recall of conditional triggers). The specialist returns a verbatim-span answer in the same schema as the baselines. M1 thus combines two architectural commitments: routing and decomposition.

M6 — Consolidated specialist (ablation). M6 is the architectural ablation control. Its prompt is the textual union of the three specialist prompts of M1, concatenated into a single agent that handles every category itself, with no router, no orchestrator, and no per-category branching. The category indicator is selected based on the request, but no decomposition is performed. The point of M6 is methodological: it lets us decide, in Chapter 5, whether any benefit M1 enjoys over B1 is attributable to architectural decomposition or merely to the more elaborated prompt content that the specialists carry. Concretely, M1 vs. M6 isolates "architecture", and M6 vs. B1 isolates "prompt content".

4.4 Implementation Stack

Orchestration. All configurations are implemented as LangGraph `StateGraphs` [Lan24a], which let us express the orchestrator/router/specialist topology as typed nodes with declared transitions. Each node consumes and emits a structured run-state object

containing the contract, the category, intermediate decisions (e.g. classifier verdict, routed specialist), and the in-progress answer. The graph topology is the executable representation of the architecture diagrams in Section 4.1.

Model client. A unified multi-provider model client wraps Anthropic, OpenAI, Google Vertex AI, and Ollama under one synchronous and one streaming interface. The client supports 19 models in total but only the 11 listed in Section 3.6 are used in official runs. Per call it records prompt tokens, completion tokens, cache-read tokens (where supported), wall-clock latency, and provider-billed cost. These diagnostics are written to the trace and aggregated downstream.

Prompt management. Prompts are stored in a YAML registry with a stable schema. The registry has two roots, `prompts/baselines/` (B1, B4) and `prompts/specialists/` (the three M1 specialists, the M6 consolidated prompt, the M0 classifier and extractor), and a separate `indicators/` directory holding the per-category indicator strings shared across configurations. The registry is loaded once per run and version-pinned in the run metadata.

Observability. Every model call is traced via Langfuse [Lan24b] with a structured trace format that records the configuration ID, the model identifier, the prompt template version, the input tokens, the output tokens, the parsed answer, and the routing or classification decision (where applicable). For M1, the trace records the orchestrator decision, the routed specialist, and the specialist's answer in a parent-child trace structure. We measured trace completeness (fraction of model calls for which all expected trace fields were populated) at 95.2% for M1 across the official runs; the remaining 4.8% are largely due to provider-side trace truncations on long contracts and are flagged in the run logs.

Data and persistence. The CUAD loader implements the 100,000-character filter, the natural-prior sampler, and the tier partition of Section 3.2. It is a deterministic component keyed by `seed=42`. Per-run outputs are persisted as JSONL files containing one record per (sample, configuration, model) prediction with full trace identifiers, allowing the statistical pipelines (Section 3.5) to be re-run from raw logs.

Language and tooling. The codebase is Python, managed with `uv`, and the statistical pipelines are built on `pandas` [The20], `numpy` [Har+20], and `scipy` [Vir+20]. Bootstrap resampling uses `numpy` random generators with run-level seeds for reproducibility.

4.5 Traceability and Auditability

A property of multi-stage agentic systems that is frequently advertised but rarely characterised is *auditability*: the claim that decomposition produces inspectable artefacts that enable reviewer trust. We treat auditability as a system property of the implementation rather than as a hypothesis to be tested, and document here what the system records and what trade-offs that recording entails. This section addresses Sub-RQ3 of Chapter 1.

For each clause-level decision, the system captures: (i) the configuration ID and model identifier, (ii) the prompt template version and the rendered prompt, (iii) the model's full completion, (iv) the parsed final answer with the parser version, (v) the routing decision (M1) or classification verdict (M0) where applicable, and (vi) the per-call token, cost, and latency diagnostics. M1 traces are structured as parent-child relationships: the orchestrator's routing call is the parent, and the routed specialist's call is its child. M0 traces analogously expose the classifier as parent and the extractor (when invoked) as child.

This trace structure supports three concrete auditability properties. First, *decision attribution*: any final answer can be traced to the specific agent that produced it and the specific prompt version that elicited it. Second, *counterfactual replay*: because prompts and model identifiers are pinned per call, re-running the trace with a different model is mechanical, which lets a reviewer ask "would this contract have been flagged by a stronger model?" without re-running the full pipeline. Third, *rejection forensics*: when M0 silently returns "no related clause" on a category, the classifier's verdict is recorded explicitly, which makes the failure mode of Section 5.5.1 (over-rejection) directly visible at audit time.

The trade-offs are substantive. Verbose traces increase storage cost (the JSONL persistence layer is on the order of gigabytes per full sweep) and, when chains-of-thought are recorded, can themselves contain sensitive contract content; we therefore distinguish between trace fields that are safe to surface to operators (decisions, token counts) and those that should be access-controlled (full prompts and completions). Auditability is also a property of the *system*, not of the underlying model: the trace is faithful to the call graph but does not certify that the model's stated reasoning is its actual reasoning.

4.6 Experimental Infrastructure

Loader and sampling. The CUAD loader is the single source of truth for which contracts and which categories enter a run. It enforces the 100,000-character filter, applies the natural-prior sampler, and emits the tier label for every sample. For the official runs the loader produces approximately 2,999 samples per run and is identical across configurations, ensuring that pairing in Chapter 5's statistical pipeline is well-defined.

Statistical aggregation. A separate aggregation pipeline reads the per-run JSONL outputs, computes the per-sample correctness vectors, and emits the McNemar contingency tables, the paired bootstrap distributions, and the BH-corrected p-value tables. Aggregation is deterministic given the JSONL inputs and a fixed bootstrap seed. The pipeline emits machine-readable summaries that feed both the thesis tables and the dashboard.

Dashboard. A Next.js dashboard is provided for interactive exploration of experiment outputs, including per-model F_2 comparisons, per-tier breakdowns, per-call cost and latency views, and trace inspection by sample ID. The dashboard is not part of the statistical pipeline but is the primary interface used during error analysis and was the channel through which the over-extraction failure mode (Chapter 5) was first surfaced visually.

Test suite. An 81-test suite covers the metric implementations (F_1/F_2, Jaccard, laziness, over-extraction), the CUAD loader (filter, sampling determinism, tier assignment), the agent topologies (B1, B4, M0, M1, M6 each have at least one end-to-end test against a stub model), and the statistical pipelines (McNemar exact, paired bootstrap, BH correction, intersection logic for partial runs). The full suite passes prior to every official run. Notebook artefacts (`01_data_exploration.ipynb`, `02_workflow_test.ipynb`, `03_baseline_calibration.ipynb`, `04_multiagent_experiment.ipynb`) are retained as reproducible exploratory entry points.

Reproducibility. Reproducibility is supported by three commitments: deterministic sampling under `seed=42`; explicit version pinning of model identifiers in the run metadata; and persistence of the full trace bundle (prompts, completions, parsed

answers, diagnostics) so that downstream statistics can be regenerated from raw logs without rerunning models.

5 Results

5.1 Run Coverage and Sample Accounting

Table 5.1 reports the per-run sample count for all 55 (model, configuration) cells. Two runs were partial: Claude Sonnet 4 on B1 (n=2,640) and GPT-4.1 on B1 (n=2,528), attributable to upstream sampling interruptions. Every statistical test uses the paired intersection of sample IDs for the specific configuration pair being compared; intersection sizes are reported individually. All other cells reach the target n=2,999.

Model	**B1**	**B4**	**M0**	**M1**	**M6**	**Total**	**B1 cost ($)**
Claude Haiku 4.5	2,999	2,999	2,999	2,999	2,999	14,995	36.70
Claude Opus 4.6	2,999	2,999	2,999	2,999	2,999	14,995	185.02
Claude Sonnet 4	2,640*	2,999	2,999	2,999	2,999	14,636	94.04
Claude Sonnet 4.6	2,999	2,999	2,999	2,999	2,999	14,995	111.44
Gemini 2.5 Flash	2,999	2,999	2,998	2,999	2,999	14,994	10.64
Gemini 2.5 Pro	2,999	2,999	2,999	2,999	2,999	14,995	43.39
Gemini 3 Flash	2,997	2,985	2,984	2,999	2,999	14,964	16.55
Gemini 3.1 Pro	2,994	2,976	2,978	2,999	2,998	14,945	65.75
GPT-4.1	2,528*	2,999	2,786	2,999	2,978	14,290	38.07
GPT-4.1 Mini	2,999	2,994	2,887	2,999	2,987	14,866	13.17
GPT-4.1 Nano	2,998	2,998	2,999	2,999	2,975	14,969	3.17
Column total	32,151	32,946	32,627	32,989	32,931	**163,644**	

Table 5.1: Coverage matrix: per-run sample count and B1 provider cost. *Partial run (upstream sampling interruption); paired tests use the intersection of sample IDs.

Of 164,941 total JSONL records across all 55 runs, 163,644 (99.21%) were analysed after dropping 1,297 records (0.79%) with malformed classification tokens, concentrated in B1 (838 drops) and M0 (362 drops). The natural positive rate across the retained set is 30.1%, confirming the sampler achieved its target class prior.

5.2 H1 — Multi-Agent vs. Single-Agent Baselines

Table 5.2 reports the per-model F_2 for all five configurations. Pooled means are: $\overline{F_2}(\text{B1}) = 0.801$, $\overline{F_2}(\text{B4}) = 0.796$, $\overline{F_2}(\text{M0}) = 0.794$, $\overline{F_2}(\text{M1}) = 0.790$, $\overline{F_2}(\text{M6}) = 0.790$, a range of 0.011 points. No configuration dominates uniformly.

Model	F_2(B1)	F_2(B4)	F_2(M0)	F_2(M1)	F_2(M6)
Claude Haiku 4.5	0.783	**0.849**	0.792	0.835	0.822
Claude Opus 4.6	0.863	**0.865**	0.864	0.860	0.846
Claude Sonnet 4	0.851	0.851	**0.877**	0.826	0.842
Claude Sonnet 4.6	0.813	0.840	**0.860**	0.834	0.836
Gemini 2.5 Flash	**0.829**	0.818	0.619	0.810	0.832
Gemini 2.5 Pro	0.828	0.810	0.707	**0.834**	0.816
Gemini 3 Flash	0.786	0.580	**0.822**	0.747	0.609
Gemini 3.1 Pro	**0.861**	0.843	0.843	0.840	0.854
GPT-4.1	**0.868**	0.852	0.828	0.777	0.853
GPT-4.1 Mini	0.837	0.841	**0.862**	0.777	0.833
GPT-4.1 Nano	0.488	0.612	**0.659**	0.547	0.545
Mean	0.801	0.796	0.794	0.790	0.790

Table 5.2: F_2 by model and configuration. Bold: best configuration per model. Configurations are ordered by pooled mean F_2. Pooled means span a 0.011-point band.

Table 5.3 reports McNemar exact p-values and Benjamini–Hochberg results for the 33 comparisons (M0 vs. B1, M1 vs. B1, M6 vs. B1 per model) constituting the H1 family, together with paired-bootstrap ΔF_2 and 95% CIs.

Figure 5.1 summarises the family-level interaction by showing mean F_2 per (provider family, configuration) cell.

The family interaction is pronounced: the Anthropic family achieves its peak under the B4 chain-of-thought baseline (mean $F_2 = 0.851$), the Google family under B1 (0.826), and the OpenAI family under M0 (0.783). All three Gemini 2.5 runs with M0 show cascading false negatives (Section 5.5.1), which suppresses the Google M0 cell to 0.748.

H1 verdict. The strict null — no multi-agent configuration outperforms the strongest single-agent baseline on F_2 pooled across all eleven models — is not rejected. The 0.011-point range across all five configurations is within sampling noise for any single model. However, 10 of 11 BH-corrected McNemar tests for M0 vs. B1 are significant, and family-conditional effects are real and large (e.g. Gemini 2.5 Flash drops 0.210 F_2

Model	ΔF_2(M0−B1)	CI_{95}	BH	ΔF_2(M1−B1)	CI_{95}	BH
Claude Haiku 4.5	+0.009	[+.050, +.068]	✓	+0.052	[+.060, +.077]	✓
Claude Opus 4.6	+0.001	[+.022, +.036]	✓	−0.004	[+.018, +.031]	✓
Claude Sonnet 4	+0.025	[+.057, +.075]	✓	−0.025	[+.024, +.043]	✓
Claude Sonnet 4.6	+0.047	[+.057, +.072]	✓	+0.021	[+.038, +.055]	✓
Gemini 2.5 Flash	−0.210	[−.045, −.015]	✓	−0.020	[+.021, +.038]	✓
Gemini 2.5 Pro	−0.121	[+.011, +.035]	✓	+0.006	[+.019, +.034]	✓
Gemini 3 Flash	+0.037	[+.043, +.063]	✓	−0.039	[+.010, +.032]	✓
Gemini 3.1 Pro	−0.018	[+.012, +.026]	✓	−0.021	[−.005, +.006]	×
GPT-4.1	−0.040	[+.040, +.061]	✓	−0.091	[+.022, +.041]	✓
GPT-4.1 Mini	+0.025	[+.014, +.027]	×	−0.060	[−.017, −.001]	×
GPT-4.1 Nano	+0.171	[+.067, +.100]	✓	+0.059	[+.024, +.067]	✓

Table 5.3: H1 McNemar exact tests with BH correction (α=0.05, m=33). ΔF_2 is the observed difference from Table 5.2; CI_{95} is the paired-bootstrap interval (B=300). ✓: BH-reject H_0.

points under M0 vs. B1). H1 is therefore conditionally rejected in the negative direction for Gemini models under M0, and conditionally supported for some Anthropic and OpenAI models.

5.3 H2 — Rare-Category Gap

Table 5.4 reports per-tier F_2 for all models under B1, M0, and M1 (the three configurations most relevant to H2). The rare–common gap $g = F_2^{\mathrm{com}} - F_2^{\mathrm{rare}}$ measures how severely each configuration penalises low-prevalence categories.

Under M0, the cascade on Gemini models (Section 5.5.1) collapses F_2^r by up to 0.338 points (Gemini 2.5 Flash: 0.719 → 0.381), inflating the mean gap. Under M1, the rare-tier F_2 degrades severely on the GPT family (GPT-4.1: 0.704 → 0.430; GPT-4.1 Mini: 0.717 → 0.529; GPT-4.1 Nano: 0.253 → 0.167), pointing to specialist over-specialisation on rare categories.

H2 verdict. Rejected. Multi-agent configurations do not reduce the rare–common F_2 gap relative to B1. M0 widens the gap to 0.284 (from 0.276) and M1 widens it to 0.342. The null hypothesis ($\Delta\mathrm{Gap} \geq 0$) cannot be rejected in the favourable direction; in fact, the gap increases significantly under M1, driven by GPT-family specialist failure on rare categories.

Family	B1	B4	M0	M1	M6
Anthropic (4 models)	0.828	0.851	0.848	0.839	0.836
Google (4 models)	0.826	0.763	0.748	0.808	0.777
OpenAI (3 models)	0.731	0.768	0.783	0.700	0.744
All (11 models)	0.801	0.796	0.794	0.790	0.790

Figure 5.1: Mean F_2 per (provider family, configuration) cell. Shading: $\geq$0.84 (dark), 0.80–0.84 (medium), 0.76–0.80 (light), $<$0.76 (very light). Best configuration per family: B4 for Anthropic, B1 for Google, M0 for OpenAI.

5.4 H3 — Architecture vs. Prompts (M1 vs. M6)

Table 5.5 reports the M1–M6 contrasts. The routing accuracy column confirms that all 11 models achieve $\geq$99.03% routing accuracy, with 10 of 11 at 100% (only GPT-4.1 Nano at 99.03%, i.e., 29 mis-routes out of 2,999).

Figure 5.2 shows M1 routing accuracy per model.

Near-perfect routing accuracy is in fact evidence *against* the diagnostic value of routing: the task reduces to a category-name lookup with three disjoint, descriptively-named specialist domains. A deterministic dictionary would achieve the same outcome at zero LLM cost.

Despite identical routing, M1 and M6 are statistically distinguishable on F_2 for all 11 models after BH correction. The direction, however, is inconsistent: M1 outperforms M6 on 5 models (Haiku 4.5, Opus 4.6, Gemini 2.5 Pro, Gemini 3 Flash, GPT-4.1 Nano), and M6 outperforms M1 on 6 models (Sonnet 4, Sonnet 4.6, Gemini 2.5 Flash, Gemini 3.1 Pro, GPT-4.1, GPT-4.1 Mini). The largest absolute differences are Gemini 3 Flash ($+0.138$ for M1) and GPT-4.1 (-0.076 for M6), suggesting a model-capacity interaction: lighter/flash models benefit from the smaller individual specialist prompts, while capable heavy models handle the consolidated prompt more effectively.

H3 verdict. Rejected. M1 and M6 are statistically distinguishable on F_2 for all 11 models ($p < 0.05$ BH-corrected). However, the direction is not consistent across model families. There is no systematic benefit of the routing architecture over the consolidated specialist prompt. The pooled means coincide ($\overline{F_2} = 0.790$ for both), and the per-model differences are explained by a model-capacity interaction rather than by routing contribution.

	B1 (baseline)				M0			M1		
Model	F_2^c	F_2^m	F_2^r	g	F_2^c	F_2^r	g	F_2^c	F_2^r	g
Claude Haiku 4.5	.926	.731	.597	.329	.770	.696	.074	.950	.654	.296
Claude Opus 4.6	.965	.815	.746	.219	.937	.735	.202	.966	.706	.260
Claude Sonnet 4	.958	.791	.714	.245	.940	.730	.211	.948	.691	.257
Claude Sonnet 4.6	.947	.746	.678	.269	.941	.721	.220	.928	.683	.245
Gemini 2.5 Flash	.935	.779	.719	.216	.725	.381	.344	.946	.668	.277
Gemini 2.5 Pro	.958	.768	.690	.268	.919	.403	.515	.969	.711	.258
Gemini 3 Flash	.950	.692	.627	.324	.944	.590	.355	.923	.577	.346
Gemini 3.1 Pro	.966	.807	.741	.225	.962	.672	.289	.963	.701	.262
GPT-4.1	.950	.841	.704	.246	.856	.677	.179	.964	.430	.533
GPT-4.1 Mini	.913	.812	.717	.196	.955	.723	.233	.950	.529	.421
GPT-4.1 Nano	.549	.518	.253	.296	.708	.412	.296	.775	.167	.608
Mean	.929	.773	.653	.276	.878	.594	.284	.940	.574	.342

Table 5.4: Per-tier F_2 for B1, M0, and M1 (c=common, m=moderate, r=rare; columns abbreviated for M0/M1). Gap $g = F_2^c - F_2^r$. Mean gap: B1=0.276, M0=0.284, M1=0.342 — multi-agent configurations widen the rare penalty.

5.5 Additional Findings

5.5.1 The Classification-Gate Rejection Cascade

Table 5.6 quantifies the M0 cascade failure. The M0 classifier emits `predicted_no_clause` (a silent empty return, bypassing the extractor) on a fraction of positive instances. The *gate-caused FN share* measures what fraction of M0 false negatives originate from this path.

The Gemini 2.5 family is the most severe case: wrong-rejection rates of 40.9% and 31.7% versus 2.7% and 1.5% for B1, with $\geq$97.8% of M0 false negatives attributable to the gate. The pattern generalises across all eleven models: on average 82.7% of M0 false negatives are gate-caused. GPT-4.1 Nano is the exception — its B1 laziness (42.0%) exceeds M0's (36.3%), because the model is too weak to reliably extract clauses even without the gate.

The cascade is more severe on rare categories: the Gemini 2.5 Flash rare-tier F_2 drops from 0.719 (B1) to 0.381 (M0), a 0.338-point loss entirely attributable to the gate. This interaction explains why M0 widens the rare–common gap (Section 5.3).

Model	F_2(M1)	F_2(M6)	ΔF_2	CI_{95}	p (McN)	BH
Claude Haiku 4.5	0.835	0.822	+0.013	[+.021, +.035]	1.7×10^{-23}	✓
Claude Opus 4.6	0.860	0.846	+0.014	[+.026, +.038]	7.8×10^{-34}	✓
Claude Sonnet 4	0.826	0.842	−0.016	[−.013, −.001]	2.5×10^{-2}	✓
Claude Sonnet 4.6	0.834	0.836	−0.002	[+.031, +.048]	1.3×10^{-45}	✓
Gemini 2.5 Flash	0.810	0.832	−0.022	[+.015, +.032]	6.8×10^{-21}	✓
Gemini 2.5 Pro	0.834	0.816	+0.018	[+.024, +.038]	2.7×10^{-36}	✓
Gemini 3 Flash	0.747	0.609	+0.138	[+.112, +.140]	1.2×10^{-129}	✓
Gemini 3.1 Pro	0.840	0.854	−0.014	[−.003, +.009]	1.4×10^{-2}	✓
GPT-4.1	0.777	0.853	−0.076	[+.015, +.024]	9.5×10^{-17}	✓
GPT-4.1 Mini	0.777	0.833	−0.056	[+.014, +.032]	5.1×10^{-26}	✓
GPT-4.1 Nano	0.547	0.545	+0.001	[+.011, +.051]	8.2×10^{-15}	✓

Table 5.5: H3: M1 vs. M6 contrasts. $\Delta F_2 = F_2(\mathrm{M1}) - F_2(\mathrm{M6})$ from Table 5.2; CI_{95} from paired bootstrap (B=300); p exact McNemar two-sided; BH at α=0.05, m=33 tests. All 11 models are BH-significant, but direction is inconsistent.

5.5.2 Cost-Efficiency Analysis

Table 5.7 reports per-run provider costs. B1 is the cheapest configuration for all eleven models. The M0 premium over B1 ranges from 1.29× (Gemini 2.5 Flash) to 2.40× (Claude Opus 4.6). M1 and M6 carry smaller premiums (1.00–1.47×), consistent with single additional orchestration calls.

Figure 5.3 plots the F_2–cost Pareto frontier across all 55 (configuration, model) cells. B1 occupies the Pareto frontier at every cost level at which it appears: no configuration is simultaneously cheaper and achieves higher F_2 for the same model. The M0 Gemini cells are dominated in both dimensions (lower F_2 *and* higher cost than B1 Gemini), making them strictly Pareto-inferior.

The cost analysis places B1 on the Pareto frontier: zero configurations are both cheaper and more accurate than B1 for any given model. The 24%–70% premium attached to multi-agent configurations is uncompensated by aggregate F_2 gains. In workflows where the per-agent trace adds operational value (obligation tracking, audit-trail generation), this premium may be acceptable; the Chapter 6 discussion elaborates these conditions.

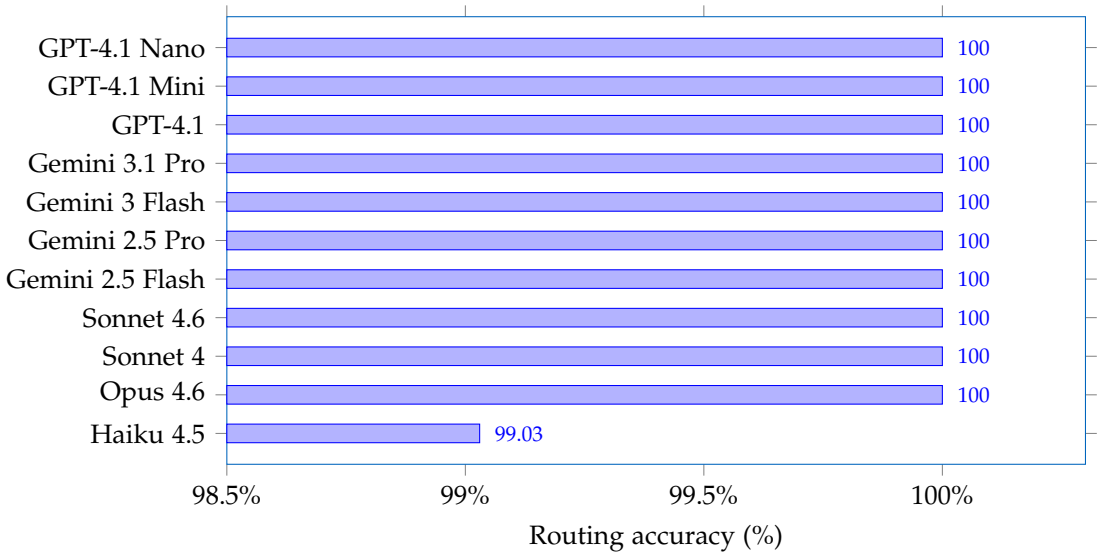


Figure 5.2: M1 router accuracy: agreement with the fixed category-to-specialist mapping. Ten of eleven models achieve 100%; GPT-4.1 Nano reaches 99.03% (29 mis-routes out of 2,999).

Model	**M0 laz.**	**B1 laz.**	**M0 FN**	**Gate-FN**	**Gate %**
Claude Haiku 4.5	16.9%	3.9%	255	210	82.4%
Claude Opus 4.6	7.6%	1.9%	142	95	66.9%
Claude Sonnet 4	7.0%	2.1%	123	87	70.7%
Claude Sonnet 4.6	10.5%	2.0%	156	131	84.0%
Gemini 2.5 Flash	40.9%	2.7%	520	509	97.9%
Gemini 2.5 Pro	31.7%	1.5%	403	394	97.8%
Gemini 3 Flash	17.9%	2.0%	221	197	89.1%
Gemini 3.1 Pro	15.6%	3.8%	191	169	88.5%
GPT-4.1	17.8%	5.6%	207	191	92.3%
GPT-4.1 Mini	11.8%	8.4%	147	103	70.1%
GPT-4.1 Nano	36.3%	42.0%	452	318	70.4%
Mean	19.5%	6.5%	256	218	82.7%

Table 5.6: M0 classification-gate rejection cascade. Laz. = laziness rate (FN fraction of positives). Gate-FN = false negatives where `predicted_no_clause` = True. Gate % = Gate-FN / M0 FN $\times$100. Gemini 2.5 Flash and Pro: $\approx$98% of M0 false negatives are gate-caused.

Model	B1 ($)	B4 ($)	M0 ($)	M1 ($)	M6 ($)
Claude Haiku 4.5	36.70	43.11	60.66	45.00	43.29
Claude Opus 4.6	185.02	213.30	302.96	215.49	211.10
Claude Sonnet 4	94.04	125.98	184.58	127.17	125.41
Claude Sonnet 4.6	111.44	129.59	184.11	137.75	128.24
Gemini 2.5 Flash	10.64	14.72	13.73	13.31	13.99
Gemini 2.5 Pro	43.39	61.77	58.69	55.57	58.21
Gemini 3 Flash	16.55	19.09	26.24	19.65	19.18
Gemini 3.1 Pro	65.75	72.46	104.57	78.73	72.69
GPT-4.1	38.07	77.88	91.50	56.11	73.68
GPT-4.1 Mini	13.17	15.78	21.42	13.22	15.78
GPT-4.1 Nano	3.17	3.64	5.10	3.64	3.63
Mean ratio vs. B1	1.00×	1.30×	1.67×	1.23×	1.27×

Table 5.7: Provider cost per run (2,999 samples). Mean ratio: average of per-model cost / B1-cost. M0 carries a 1.67× mean premium; B4 is 1.30×; M1 and M6 are ≈1.25×.

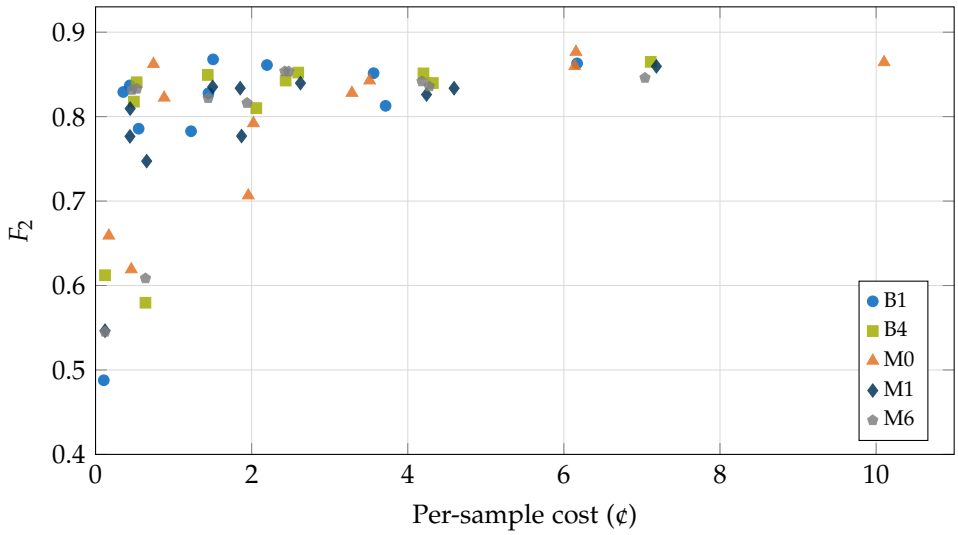


Figure 5.3: F_2 vs. per-sample provider cost for all 55 (configuration, model) cells. B1 (circle) dominates or matches every configuration at every cost level. Three M0 outliers (Gemini 2.5 Flash/Pro and Gemini 3 Flash) are dominated in both dimensions.

6 Discussion

6.1 When Multi-Agent Helps: Evidence for Pipeline Decomposition

6.2 When Multi-Agent Hurts: The LLM-Router Problem

6.3 Architecture vs Prompts: The H3 Verdict

6.4 Practitioner Perspective: Gaps Between Benchmarks and Real-World CLM

6.5 Connecting to Prior Work

6.6 Limitations

7 Conclusion and Outlook

7.1 Summary of Contributions

7.2 Answers to Research Questions

7.3 Outlook and Future Work

List of Figures

2.1 CLM lifecycle. Clause extraction, risk identification, and obligation tracking (highlighted) occur primarily in the Execute and Monitor stages. 6

2.2 Canonical hierarchical multi-agent architecture. The orchestrator coordinates; the router dispatches to one specialist per request; specialists return typed answers. Dashed arrows indicate conditional paths. 6

3.1 Message-flow diagrams for the five configurations. Input = contract text + clause-category indicator. Blue nodes: generalist LLM invocations. Orange nodes: domain-specialist prompts. $\emptyset$ = silent no-clause return. 12

5.1 Mean F_2 per (provider family, configuration) cell. Shading: $\geq$0.84 (dark), 0.80–0.84 (medium), 0.76–0.80 (light), $<$0.76 (very light). Best configuration per family: B4 for Anthropic, B1 for Google, M0 for OpenAI. 26

5.2 M1 router accuracy: agreement with the fixed category-to-specialist mapping. Ten of eleven models achieve 100%; GPT-4.1 Nano reaches 99.03% (29 mis-routes out of 2,999). 29

5.3 F_2 vs. per-sample provider cost for all 55 (configuration, model) cells. B1 (circle) dominates or matches every configuration at every cost level. Three M0 outliers (Gemini 2.5 Flash/Pro and Gemini 3 Flash) are dominated in both dimensions. 30

List of Tables

2.1 CUAD dataset statistics [Hen+21]. 7
2.2 Main contract-analysis benchmarks. 8
2.3 Multi-agent systems for legal AI coded on five architectural dimensions. All prior systems evaluate a single architecture without an ablation. . . 8

3.1 Five evaluated configurations. M0 vs. B1 isolates the classification gate; M1 vs. M6 isolates routing and decomposition; B4 vs. B1 isolates chain-of-thought. 11
3.2 Eleven frontier proprietary models evaluated. Three families; compute-tier range covers light-to-heavy within each family. 14

4.1 The five evaluated configurations across the three design axes. 16

5.1 Coverage matrix: per-run sample count and B1 provider cost. *Partial run (upstream sampling interruption); paired tests use the intersection of sample IDs. 23
5.2 F_2 by model and configuration. Bold: best configuration per model. Configurations are ordered by pooled mean F_2. Pooled means span a 0.011-point band. 24
5.3 H1 McNemar exact tests with BH correction (α=0.05, m=33). ΔF_2 is the observed difference from Table 5.2; CI_{95} is the paired-bootstrap interval (B=300). ✓: BH-reject H_0. 25
5.4 Per-tier F_2 for B1, M0, and M1 (c=common, m=moderate, r=rare; columns abbreviated for M0/M1). Gap $g = F_2^c - F_2^r$. Mean gap: B1=0.276, M0=0.284, M1=0.342 — multi-agent configurations widen the rare penalty. 27
5.5 H3: M1 vs. M6 contrasts. $\Delta F_2 = F_2(\mathrm{M1}) - F_2(\mathrm{M6})$ from Table 5.2; CI_{95} from paired bootstrap (B=300); p exact McNemar two-sided; BH at α=0.05, m=33 tests. All 11 models are BH-significant, but direction is inconsistent. 28

5.6 M0 classification-gate rejection cascade. Laz. = laziness rate (FN fraction of positives). Gate-FN = false negatives where `predicted_no_clause` = True. Gate % = Gate-FN / M0 FN ×100. Gemini 2.5 Flash and Pro: ≈98% of M0 false negatives are gate-caused. 29
5.7 Provider cost per run (2,999 samples). Mean ratio: average of per-model cost / B1-cost. M0 carries a 1.67× mean premium; B4 is 1.30×; M1 and M6 are ≈1.25×. 30

Bibliography

[BH95] Y. Benjamini and Y. Hochberg. "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing." In: *Journal of the Royal Statistical Society, Series B (Methodological)* 57.1 (1995), pp. 289–300. DOI: 10.1111/j.2517-6161.1995.tb02031.x.

[Bog+25] T. Bogavelli et al. *AgentArch: A Comprehensive Benchmark to Evaluate Agent Architectures in Enterprise*. ServiceNow Research. Full author list at arXiv:2509.10769. 2025. arXiv: 2509.10769.

[Car+20] D. Card, P. Henderson, U. Khandelwal, R. Jia, K. Mahowald, and D. Jurafsky. "With Little Power Comes Great Responsibility." In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP 2020)*. 2020, pp. 9263–9274. DOI: 10.18653/v1/2020.emnlp-main.745. arXiv: 2010.06595.

[Cem+25] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica. *Why Do Multi-Agent LLM Systems Fail?* 2025. arXiv: 2503.13657.

[Cha+20] I. Chalkidis, M. Fergadiotis, P. Malakasiotis, N. Aletras, and I. Androutsopoulos. "LEGAL-BERT: The Muppets Straight Out of Law School." In: *Findings of the Association for Computational Linguistics: EMNLP 2020*. 2020, pp. 2898–2904. arXiv: 2010.02559.

[Cha+24] A. Chatterjee, H. S. V. N. S. K. Renduchintala, S. Bhatia, and T. Chakraborty. "POSIX: A Prompt Sensitivity Index for Large Language Models." In: *Findings of the Association for Computational Linguistics: EMNLP 2024*. 2024, pp. 14550–14565. arXiv: 2410.02185.

[Cui+24] J. Cui, M. Ning, Z. Li, B. Chen, Y. Yan, H. Li, B. Ling, Y. Tian, and L. Yuan. *ChatLaw: A Multi-Agent Collaborative Legal Assistant with Knowledge Graph Enhanced Mixture-of-Experts Large Language Model*. Version 2 (2024); v1 (2023) is the original retrieval-augmented system. 2024. arXiv: 2306.16092.

[Dev+19] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT 2019)*. 2019, pp. 4171–4186. arXiv: 1810.04805.

[Dro+18] R. Dror, G. Baumer, S. Shlomov, and R. Reichart. "The Hitchhiker's Guide to Testing Statistical Significance in Natural Language Processing." In: *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL 2018)*. 2018, pp. 1383–1392.

[EC16] European Parliament and Council of the European Union. *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data (General Data Protection Regulation)*. Official Journal of the European Union, L 119, 4.5.2016, pp. 1–88. https://eur-lex.europa.eu/eli/reg/2016/679/oj. 2016.

[EC24] European Parliament and Council of the European Union. *Regulation (EU) 2024/1689 of the European Parliament and of the Council of 13 June 2024 laying down harmonised rules on artificial intelligence (Artificial Intelligence Act)*. Official Journal of the European Union, L 2024/1689, 12.7.2024. http://data.europa.eu/eli/reg/2024/1689/oj. 2024.

[Efr79] B. Efron. "Bootstrap Methods: Another Look at the Jackknife." In: *The Annals of Statistics* 7.1 (1979), pp. 1–26. DOI: 10.1214/aos/1176344552.

[Gao+25] M. Gao, Y. Li, B. Liu, Y. Yu, P. Wang, C.-Y. Lin, and F. Lai. *Single-agent or Multi-agent Systems? Why Not Both?* 2025. arXiv: 2505.18286.

[Har+20] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant. "Array Programming with NumPy." In: *Nature* 585 (2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2.

[Hat+23] J. Hatzius, J. Briggs, D. Kodnani, and G. Pierdomenico. *The Potentially Large Effects of Artificial Intelligence on Economic Growth (Briggs/Kodnani)*. Goldman Sachs Global Economics Analyst. Published 26 March 2023. Primary report is paywalled; widely cited in *The Globe and Mail* and *Law.com*. Mar. 2023.

[Hen+21] D. Hendrycks, C. Burns, A. Chen, and S. Ball. "CUAD: An Expert-Annotated NLP Dataset for Legal Contract Review." In: *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks (NeurIPS 2021)*. 2021. arXiv: 2103.06268.

[Hon+24] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber. "MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework." In: *12th International Conference on Learning Representations (ICLR 2024)*. Oral presentation. 2024. arXiv: 2308.00352.

[KM21] Y. Koreeda and C. D. Manning. "ContractNLI: A Dataset for Document-level Natural Language Inference for Contracts." In: *Findings of the Association for Computational Linguistics: EMNLP 2021*. 2021, pp. 1907–1919. arXiv: 2110.01799.

[Lam+16] G. Lample, M. Ballesteros, S. Subramanian, K. Kawakami, and C. Dyer. "Neural Architectures for Named Entity Recognition." In: *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT 2016)*. 2016, pp. 260–270. arXiv: 1603.01360.

[Lan24a] LangChain Inc. *LangGraph: Building Stateful, Multi-Actor Applications with LLMs*. Software library. https://github.com/langchain-ai/langgraph. 2024.

[Lan24b] Langfuse GmbH. *Langfuse: Open-Source LLM Engineering Platform*. Software / documentation. https://langfuse.com/ ; https://github.com/langfuse/langfuse. 2024.

[Li+23] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem. "CAMEL: Communicative Agents for "Mind" Exploration of Large Language Model Society." In: *Advances in Neural Information Processing Systems (NeurIPS 2023)*. 2023. arXiv: 2303.17760.

[Liu+25] S. Liu, Z. Li, R. Ma, H. Zhao, and M. Du. *ContractEval: Benchmarking LLMs for Clause-Level Legal Risk Identification in Commercial Contracts*. 2025. arXiv: 2508.03080.

[LMP01] J. D. Lafferty, A. McCallum, and F. C. N. Pereira. "Conditional Random Fields: Probabilistic Models for Segmenting and Labeling Sequence Data." In: *Proceedings of the 18th International Conference on Machine Learning (ICML 2001)*. 2001, pp. 282–289.

[Mar+24] L. Martin, N. Whitehouse, S. Yiu, L. Catterson, and R. Perera. *Better Call GPT: Comparing Large Language Models Against Lawyers*. Onit AI Centre of Excellence. Vendor-funded; directional findings replicated by independent evaluations. 2024. arXiv: 2401.16212.

[McN47] Q. McNemar. "Note on the Sampling Error of the Difference between Correlated Proportions or Percentages." In: *Psychometrika* 12.2 (1947), pp. 153–157. DOI: 10.1007/BF02295996.

[Rap+25] P. Raptopoulos, G. Filandrianos, M. Lymperaiou, and G. Stamou. "PAKTON: A Multi-Agent Framework for Question Answering in Long Legal Agreements." In: *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP 2025)*. Oral presentation. 2025. arXiv: 2506.00608.

[RM05] S. Riezler and J. T. Maxwell III. "On Some Pitfalls in Automatic Evaluation and Significance Testing for MT." In: *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*. 2005, pp. 57–64.

[San25] M. Sanwal. *Layered Chain-of-Thought Prompting for Multi-Agent LLM Systems: A Comprehensive Approach to Explainable Large Language Models*. 2025. arXiv: 2501.18645.

[The20] The pandas development team. *pandas-dev/pandas: Pandas*. Zenodo. Cite the Zenodo release matching the version used in the study. 2020. DOI: 10.5281/zenodo.3509134.

[Tug+20] D. Tuggener, P. von Däniken, T. Peetz, and M. Cieliebak. "LEDGAR: A Large-Scale Multi-label Corpus for Text Classification of Legal Provisions in Contracts." In: *Proceedings of the 12th Language Resources and Evaluation Conference (LREC 2020)*. 2020, pp. 1235–1241.

[Vir+20] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors. "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python." In: *Nature Methods* 17 (2020), pp. 261–272. DOI: 10.1038/s41592-019-0686-2.

[Wan+23] S. H. Wang, A. Scardigli, L. Tang, W. Chen, D. Levkin, A. Chen, S. Ball, T. Woodside, O. Zhang, and D. Hendrycks. "MAUD: An Expert-Annotated Legal NLP Dataset for Merger Agreement Understanding." In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP 2023)*. 2023, pp. 16369–16382. arXiv: 2301.00876.

[Wan+24] J. Wang, J. Wang, B. Athiwaratkun, C. Zhang, and J. Zou. *Mixture-of-Agents Enhances Large Language Model Capabilities*. Also published at ICLR 2025. 2024. arXiv: 2406.04692.

[Wor25] World Commerce & Contracting. *Contract Management: An Overlooked Driver of Business Agility and Financial Performance*. WorldCC Whitepaper. Available at https://www.worldcc.com/Portals/IACCM/Reports/Contract%20Management%20Whitepaper.pdf. 2025.

[Wu+23] Q. Wu, G. Bansal, J. Zhang, Y. Wu, S. Zhang, E. Zhu, B. Li, L. Jiang, X. Zhang, and C. Wang. *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework*. Also published at COLM 2024. 2023. arXiv: 2308.08155.

[WY25] Z. Wang and B. L. Yuan. *L-MARS: Legal Multi-Agent Workflow with Orchestrated Reasoning and Agentic Search*. 2025. arXiv: 2509.00761.